

Espectro_p1

March 27, 2025

0.1 Astronomía Observacional

0.2 Astropy, Tutorial 1: Reducción de datos espectroscópicos Parte 1: Rastreo

María Alejandra Moreno, Cesar Mauricio Mendoza

0.3 Paso a paso de obtención de espectros usando Astropy

#Paso 1 : Examinar el espectro

Importar los paquetes para leer imágenes por medio de Python Image y utilizar numpy para operaciones.

```
[1]: from PIL import Image
import numpy as np
from astropy.modeling.models import Polynomial1D

import matplotlib.pyplot as plt
plt.rcParams['image.origin'] = 'lower'
plt.style.use('dark_background') # Optional!
```

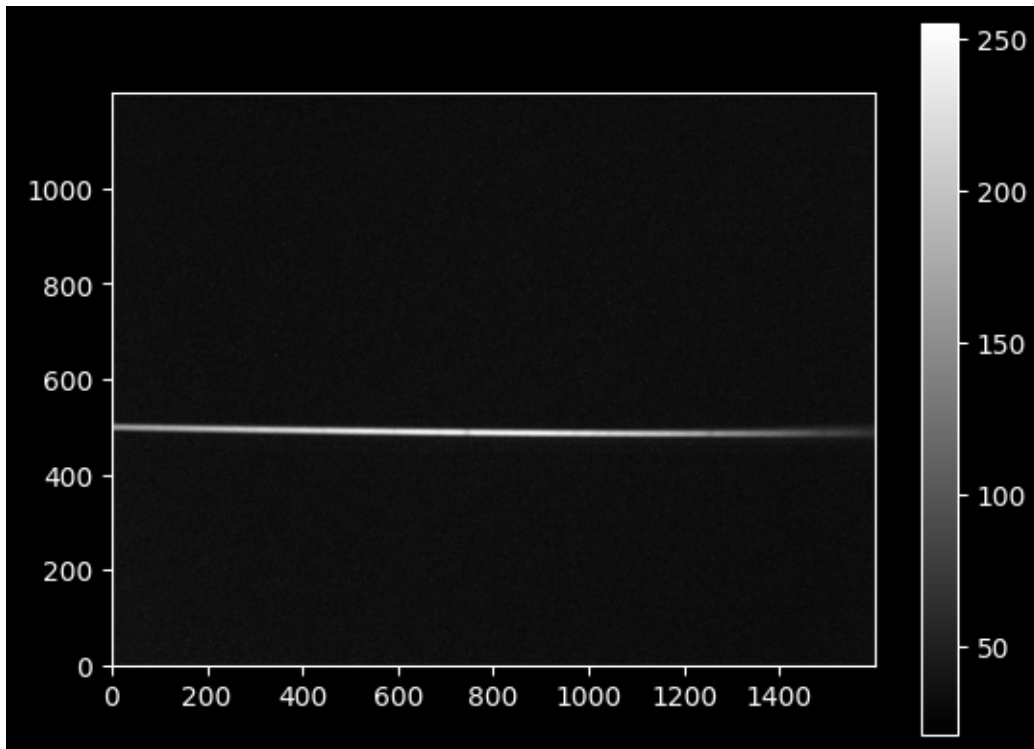
En este código se importa el módulo Image de la biblioteca PIL (Pillow), que se usa para abrir, modificar y guardar imágenes en diferentes formatos.

NumPy se usa para trabajar arreglos matemáticos.

Se importa el módulo pyplot de Matplotlib para crear gráficos y visualizar datos de manera sencilla.

Se cambia la configuración de Matplotlib para que el origen de las imágenes esté en la esquina inferior izquierda y el estilo de los gráficos para que usen un fondo oscuro.

```
[2]: spectrum_filename = "aldebaran_3s_1.bmp"
image_array_2 = np.array(Image.open('deneb_3s_13.63g_1.bmp'))
plt.imshow(image_array_2, cmap='gray')
plt.colorbar();
```



```
[3]: image_data = Image.open(spectrum_filename)
```

Se define una variable que almacena el nombre del archivo de imagen (en este caso, un archivo BMP).

Ahora la imagen un objeto de tipo `PIL.Image.Image`, lo que significa que se puede manipular la imagen usando los métodos de PIL, como cambiar su formato, tamaño o convertirla en una matriz NumPy.

Finalmente se muestra la imagen en pantalla automáticamente.

```
[4]: image_data
```

```
[4]:
```



```
[5]: image_array = np.array(image_data)
      image_array, image_array.shape
```

```
[5]: (array([[41, 43, 50, ..., 52, 28, 27],
             [33, 22, 48, ..., 37, 35, 26],
             [41, 64, 30, ..., 21, 30, 33],
             ...,
             [33, 28, 27, ..., 26, 33, 28],
             [35, 25, 23, ..., 23, 46, 46],
             [32, 35, 22, ..., 23, 40, 29]]), shape=(1200, 1600), dtype=uint8),
      (1200, 1600))
```

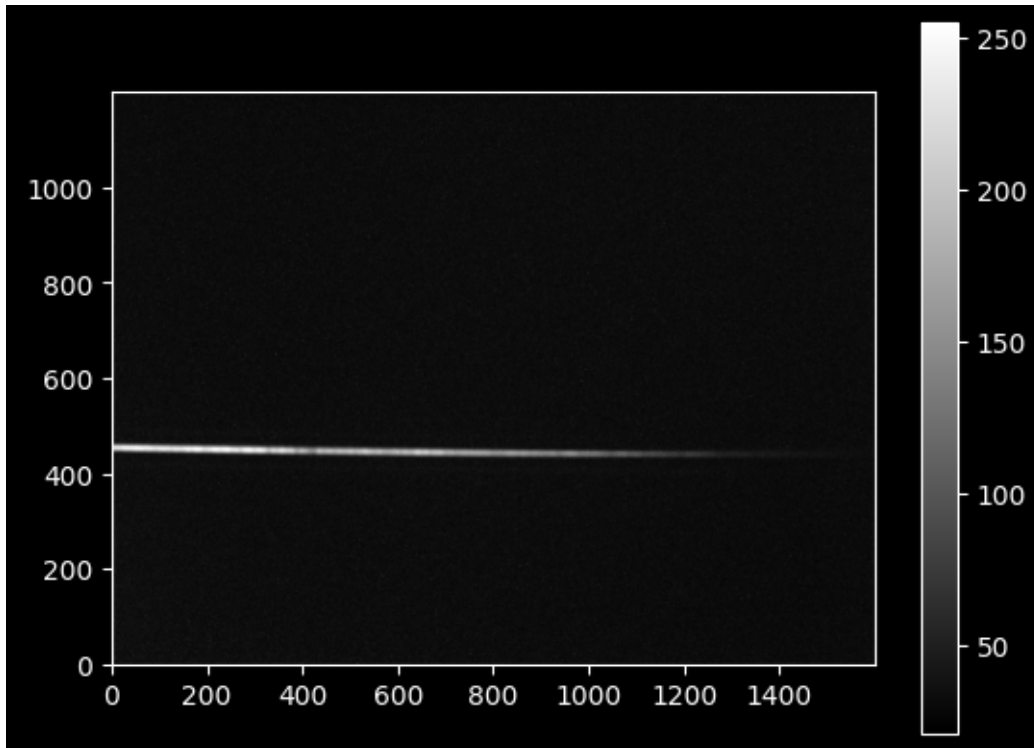
Aquí se convierte la imagen en un array NumPy. Este array representa la imagen en forma de una matriz de píxeles, que tendrá la forma (alto, ancho).

Muestra la imagen almacenada en `image_data`, `cmap='gray'` indica que la imagen debe visualizarse en escala de grises.

Se agrega una barra de colores al lado de la imagen, lo cual es útil cuando se trata de datos de intensidad o imágenes en escala de grises.

```
[6]: plt.imshow(image_data, cmap='gray')
plt.colorbar()
```

```
[6]: <matplotlib.colorbar.Colorbar at 0x7f47304184d0>
```



El objetivo es obtener un modelo $f(x)$ que describa la posición vertical de la señal en la imagen, Comenzaremos asumiendo que la dispersión de la longitud de onda se produce en la dirección X y que la dirección Y es completamente espacial.

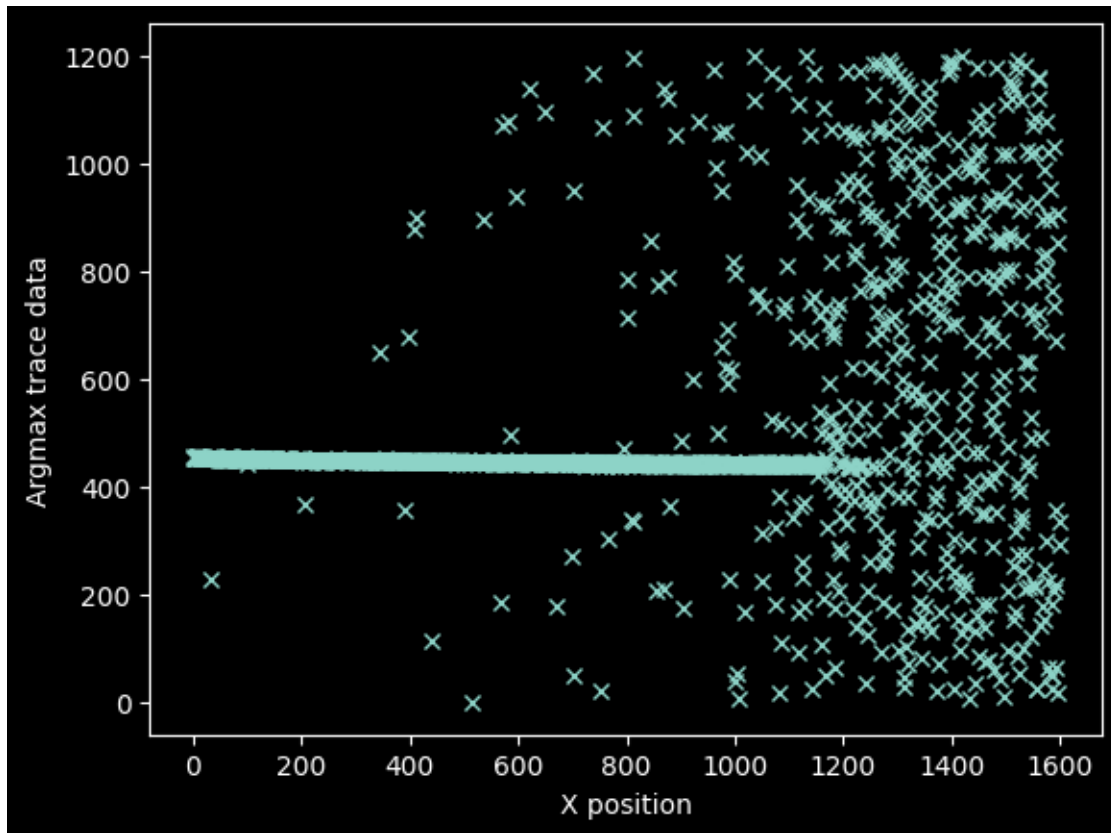
1 Paso 2: Intentar encontrar la estructura central para trazar usando argmax.

Para obtener la traza, primero medimos el valor Y en cada valor X. Comenzaremos con el enfoque trivial de usar argmax:

```
[7]: yvals = np.argmax(image_data, axis=0)
xvals = np.arange(image_data.width)
```

Este código está tratando de encontrar la posición de la señal en función de la coordenada X, basándose en la imagen del espectro.

```
[8]: plt.plot(xvals, yvals, 'x')
plt.ylabel("Argmax trace data")
plt.xlabel("X position");
```



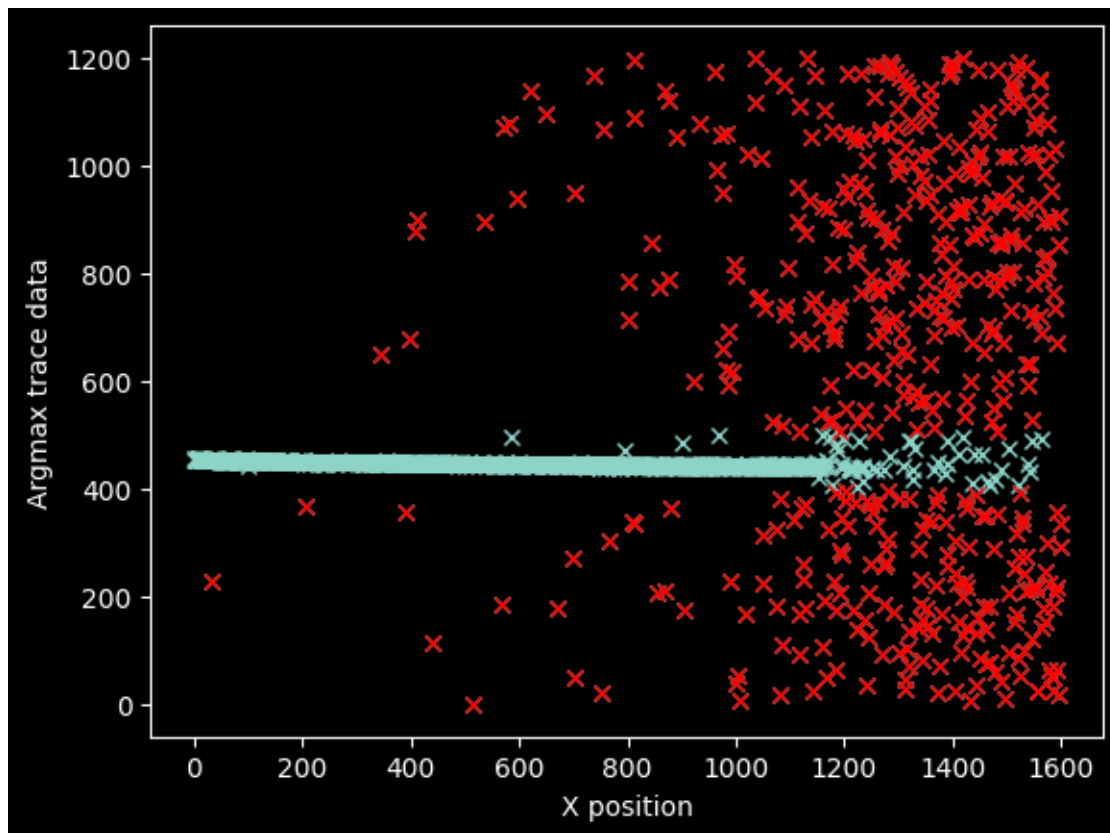
`np.argmax(image_data, axis=0)`: Encuentra el índice del valor máximo a lo largo del eje vertical (eje 0) para cada columna de la matriz `image_data`.

Este código genera una “traza de máximos” de la imagen. Es decir, muestra la posición del píxel más brillante (o de mayor intensidad) en cada columna de la imagen, lo que podría ayudar a detectar contornos o formas dentro de la imagen.

```
[9]: bad_pixels = (yvals < 400) | (yvals > 500)
```

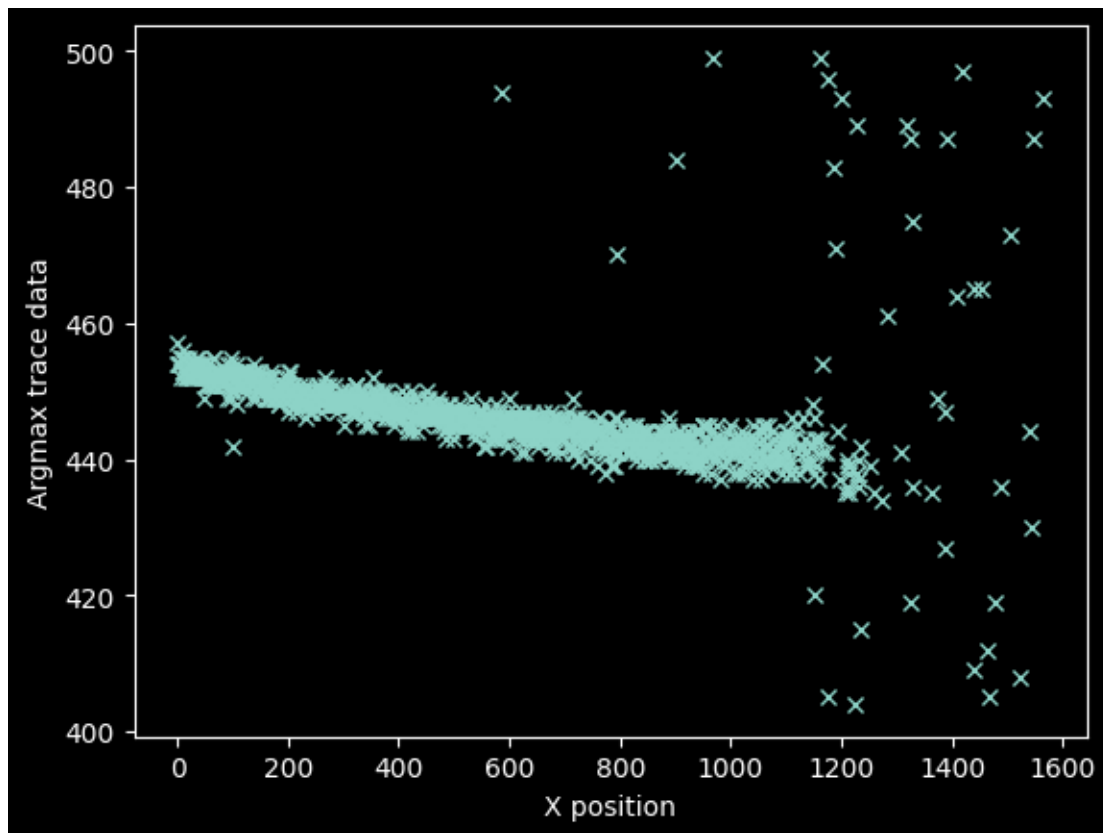
El objetivo de esta línea de código es descartar valores fuera de un umbral esperado, es decir fuera del trazado de la estrella.

```
[10]: plt.plot(xvals, yvals, 'x')
plt.plot(xvals[bad_pixels], yvals[bad_pixels], 'rx')
plt.ylabel("Argmax trace data")
plt.xlabel("X position");
```



Este fragmento de código visualiza la traza de máximos en una imagen, resaltando en rojo los píxeles que están fuera del rango permitido.

```
[11]: plt.plot(xvals[~bad_pixels], yvals[~bad_pixels], 'x')  
      plt.ylabel("Argmax trace data")  
      plt.xlabel("X position");
```

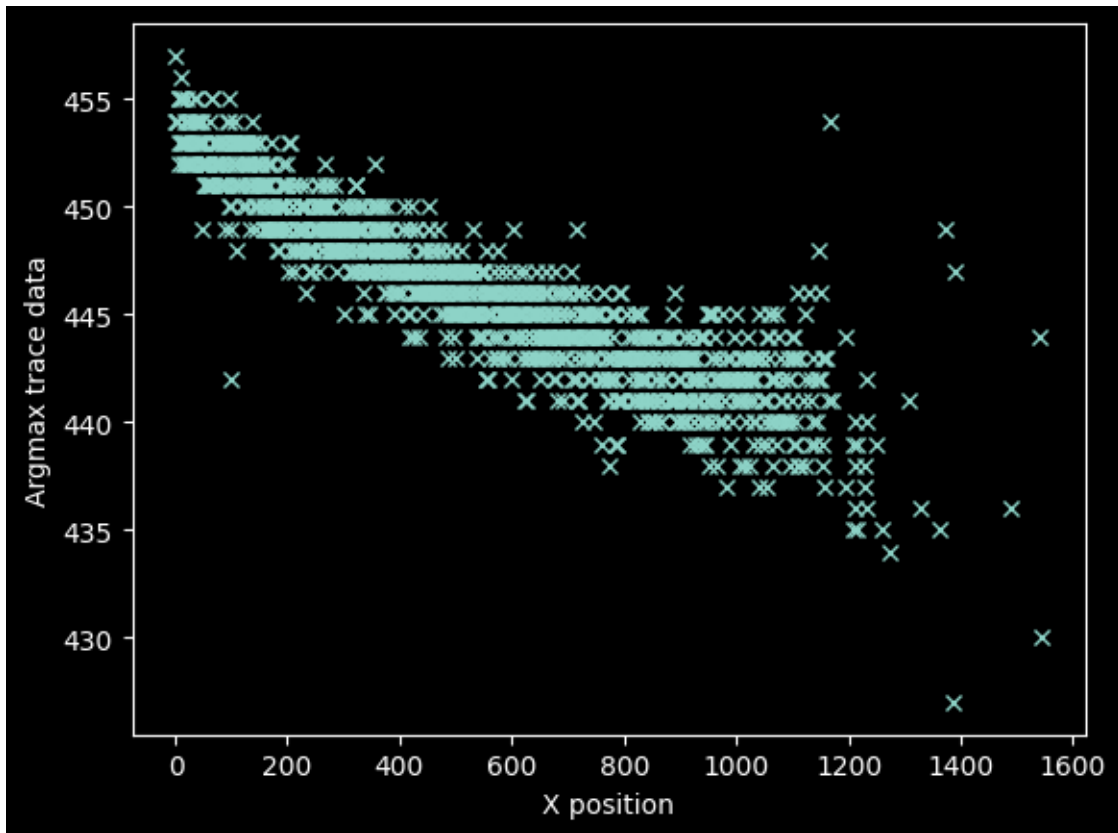


Esta parte visualiza solo los píxeles buenos (dentro del rango permitido) en la traza de máximos de la imagen, permitiendo ver solo los valores válidos, proporcionando una mejor interpretación de la traza de máximos.

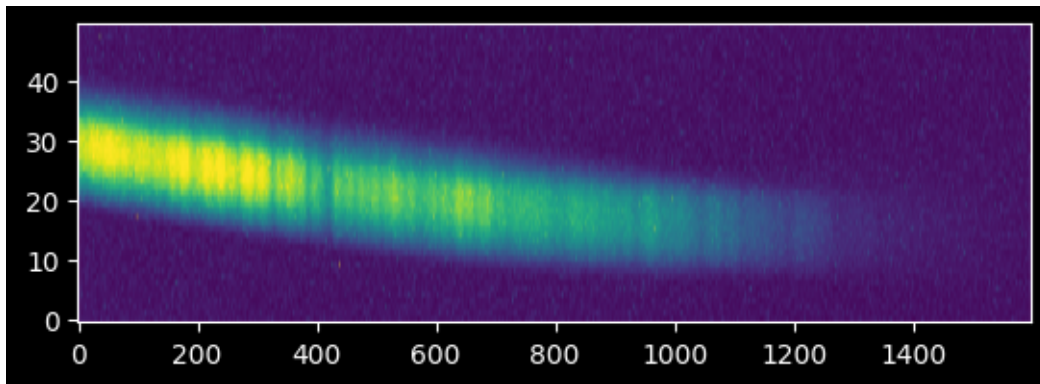
```
[12]: bad_pixels = (yvals < 425) | (yvals > 460)
```

Esta línea define una nueva condición para detectar píxeles “malos” en la traza de máximos de la imagen, usando un umbral más estricto que la versión anterior (400-500), para enfocar aún más al trazado de la estrella.

```
[13]: plt.plot(xvals[~bad_pixels], yvals[~bad_pixels], 'x')
plt.ylabel("Argmax trace data")
plt.xlabel("X position");
```

```
[14]: plt.imshow(image_array[425:475,:])
      plt.gca().set_aspect(10)
```



```
plt.imshow(image_array[425:475,:])
```

Extrae una porción de la imagen desde la fila 425 hasta 475 (50 filas en total).

`plt.gca().set_aspect(10)` Ajusta la relación de aspecto del gráfico. 10 significa que la altura de los píxeles se estira 10 veces más que su ancho, haciendo la imagen más alargada verticalmente.

2 Paso 2.b : Utiliza el análisis de momentos para extraer una estructura central a trazar.

Podemos usar momentos para obtener una estimación diferente, posiblemente mejor, de la ubicación del centro de la traza. La ventaja del análisis de momentos es que utilizamos todos los datos para estimar la posición vertical, no solo el valor más brillante, que es el que utilizamos anteriormente.

Tenga en cuenta que debemos restar el fondo para evitar un sesgo hacia el centro, por lo que utilizamos la mediana de toda la imagen como estimación del fondo.

(El momento de primer orden es la posición media ponderada por la intensidad,

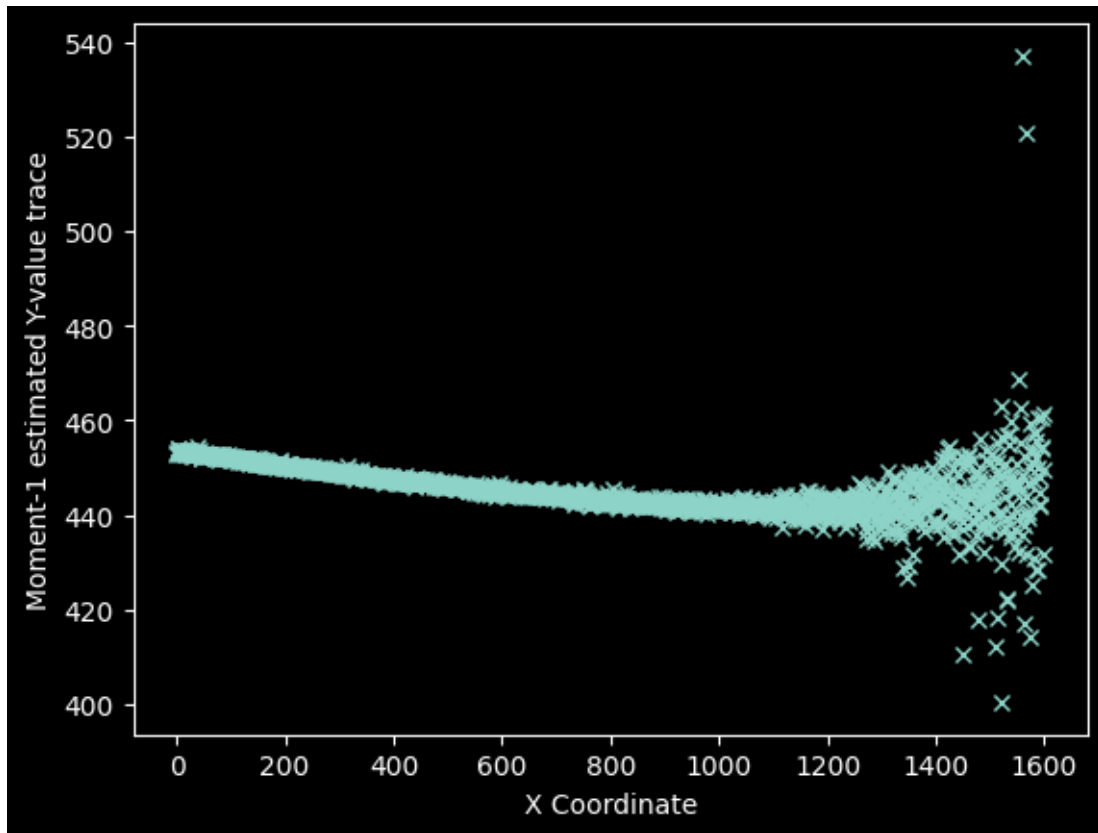
$$m_1 = \frac{\sum_i x_i f(x_i)}{\sum_i f(x_i)}$$

Donde x_i es la posición y $f(x_i)$ es la intensidad en esa posición. $f(x_i)$ debe ser cero en la región libre de señal para m_1 para devolver una estimación precisa de la ubicación del pico)

```
[15]: yaxis = np.repeat(np.arange(425, 475)[: ,None],  
                      image_array.shape[1], axis=1)  
background = np.median(image_array)  
# moment 1 is the data-weighted average of the Y-axis coordinates  
weighted_yaxis_values = np.average(yaxis, axis=0,  
                                   weights=image_array[425:475,:] - background)
```

Usamos un recorte alrededor de la línea trazada, por lo que los valores Y provienen de ese recorte. El comando `repeat` se usa para extender los valores de posición del eje Y, que son 425, 426, ... 475, A lo largo de la dirección X. La indexación con `[: ,None]` agrega un eje “ficticio” a lo largo de la segunda dimensión (x). Luego, `repeat` copia los valores del eje Y. La matriz resultante tiene la misma forma que nuestra matriz de pesos, Que es `image_array[425:475, :]` menos la mediana.

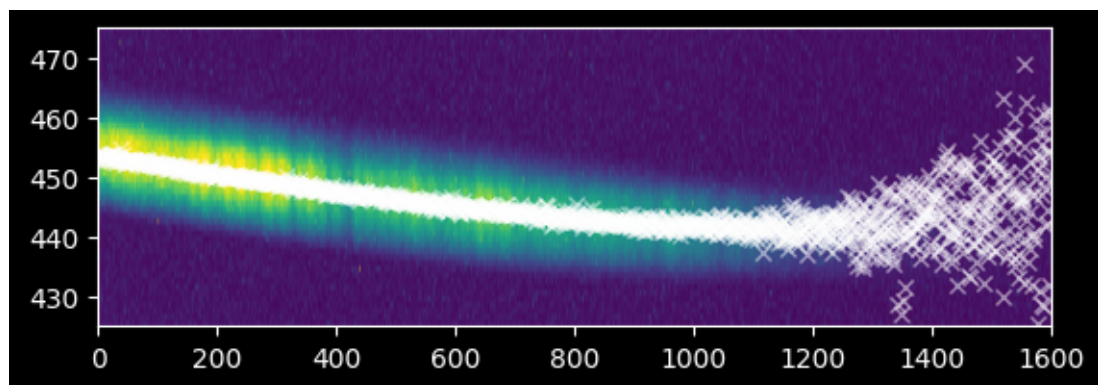
```
[16]: plt.plot(xvals, weighted_yaxis_values, 'x')  
plt.xlabel("X Coordinate")  
plt.ylabel("Moment-1 estimated Y-value trace");
```



Muestra cómo cambia la posición estimada de la línea trazada en la imagen a lo largo del eje X.

Se basa en la distribución de intensidad de la imagen, destacando la posición promedio de la señal en cada X.

```
[17]: plt.imshow(image_array[425:475,:],
                extent=[0,image_array.shape[1],425,475])
plt.gca().set_aspect(10) # Estiramos la imagen 10x en la dirección y
plt.plot(xvals, weighted_yaxis_values, 'wx', alpha=0.5)
plt.axis((0,1600,425,475));
```



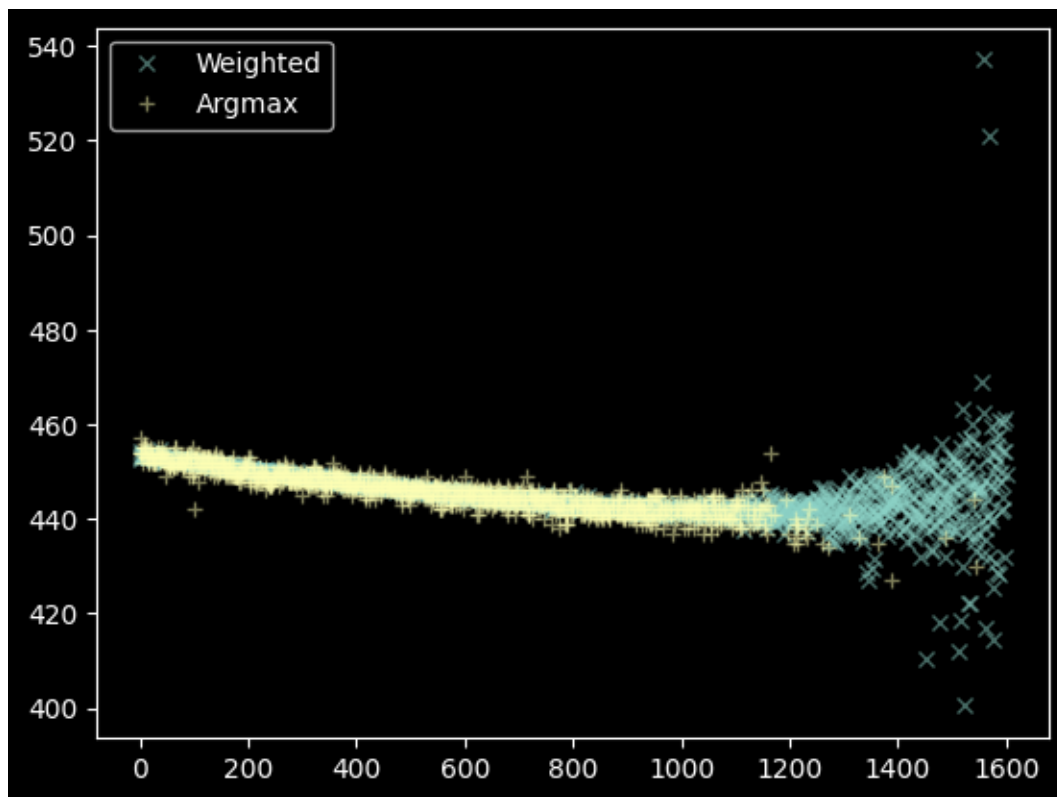
Se muestra una franja de la imagen (entre $Y = 425$ y $Y = 475$).

Los puntos blancos marcan la posición estimada de la señal en cada X.

La imagen está estirada en Y para facilitar la visualización.

También podemos comparar los enfoques argmax y ponderado. Coinciden bien en $x < 1200$, pero simplemente hay más puntos del enfoque ponderado en $x > 1200$.

```
[18]: plt.plot(xvals, weighted_yaxis_values, 'x', label="Weighted", alpha=0.5)
plt.plot(xvals[~bad_pixels], yvals[~bad_pixels], '+', label="Argmax", alpha=0.5)
plt.legend(loc='best');
```



Compara el método Argmax (máximo absoluto) con el método Weighted (Momento-1).

Si los dos métodos coinciden bien, significa que la señal es fuerte y clara.

Si hay desviaciones, puede indicar ruido, variaciones en la intensidad, o problemas con el método de Argmax.

Ese es un buen conjunto de datos; usaremos los momentos en lugar del argmax. Sin embargo, aún quedan algunos datos por destacar.

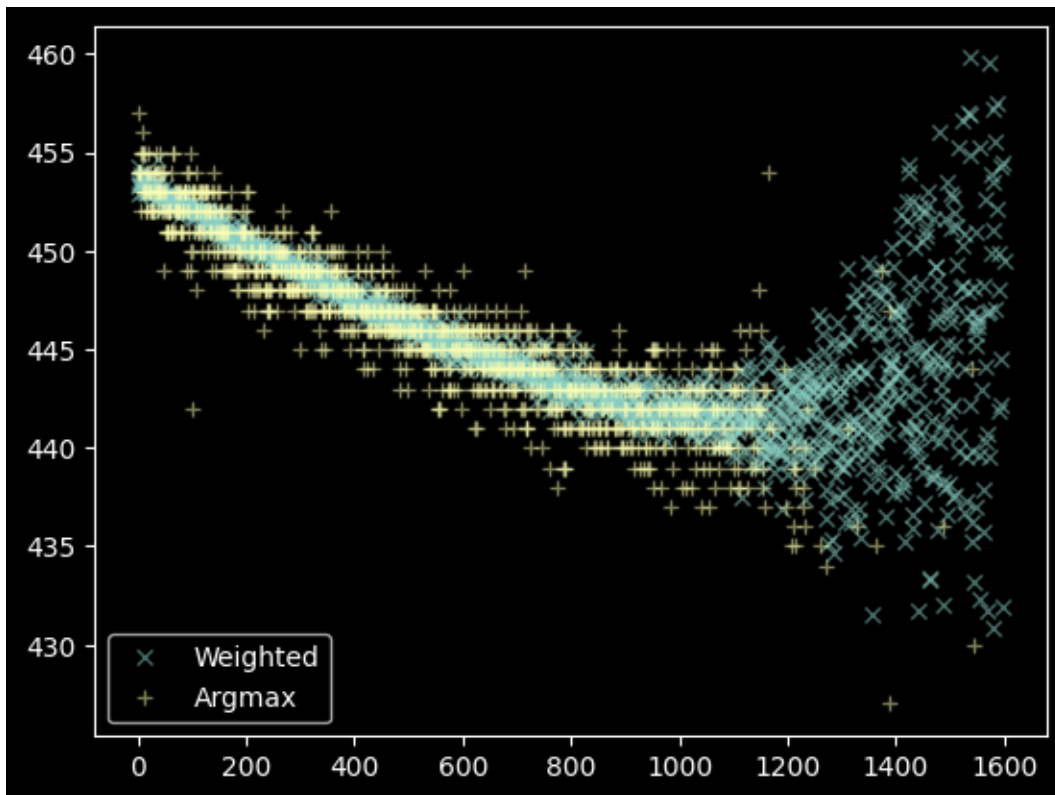
```
[19]: bad_moments = (weighted_yaxis_values > 460) | (weighted_yaxis_values < 430)
```

Comparación visual de los valores Y obtenidos con dos métodos diferentes.

Se filtran valores anómalos para mejorar la precisión del análisis.

Muestra si hay desviaciones entre los dos métodos debido a ruido o diferencias en la intensidad.

```
[20]: plt.plot(xvals[~bad_moments], weighted_yaxis_values[~bad_moments], 'x',  
             label="Weighted", alpha=0.5)  
plt.plot(xvals[~bad_pixels], yvals[~bad_pixels], '+', label="Argmax", alpha=0.5)  
plt.legend(loc='best');
```



2.1 Paso 3: Ajustar el perfil de la traza.

Queremos un modelo $f(x)$ que dé el valor “y” del centroide en función de x.

```
[21]: from astropy.modeling.polynomial import Polynomial1D  
from astropy.modeling.fitting import LinearLSQFitter
```

Estas dos importaciones permiten construir y ajustar modelos polinómicos a datos experimentales o simulados

```
[22]: polymodel = Polynomial1D(degree=2)
linfitter = LinearLSQFitter()
fitted_polymodel = linfitter(polymodel, xvals[~bad_moments],
↪ weighted_yaxis_values[~bad_moments])
```

Ajustamos un polinomio de segundo orden

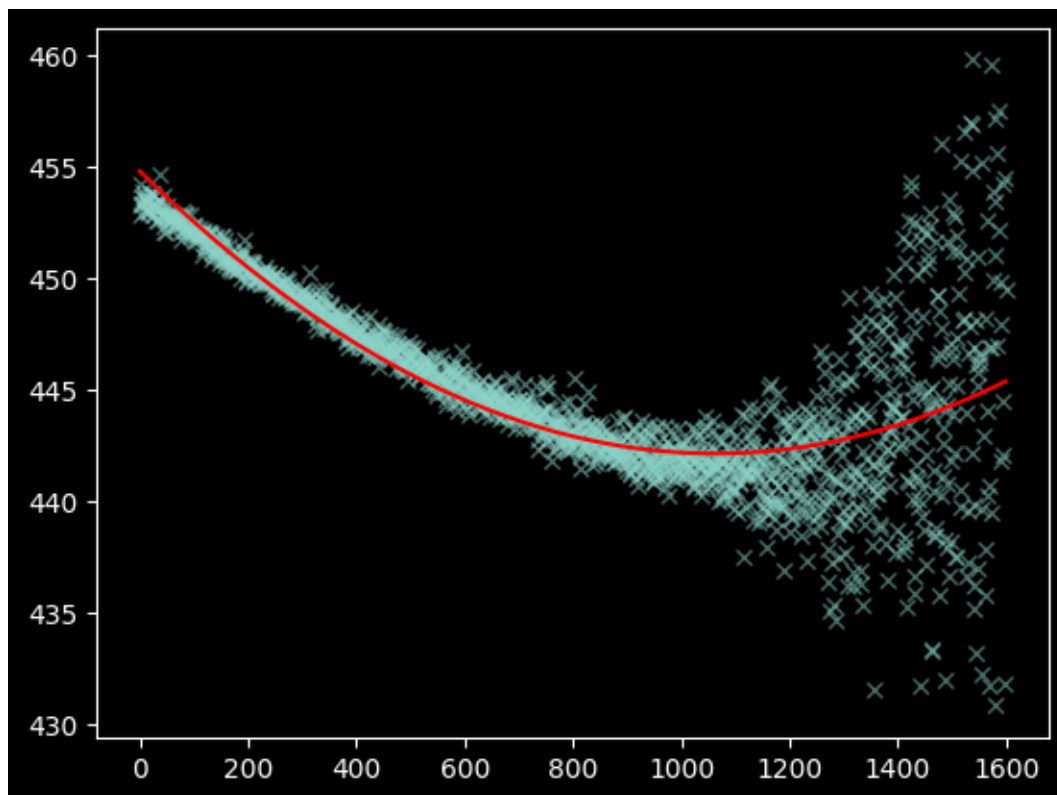
```
[23]: fitted_polymodel
```

```
[23]: <Polynomial1D(2, c0=454.80138106, c1=-0.02381282, c2=0.00001121)>
```

$$f(x) = 0.00001121x^2 - 0.02381282x + 454.80138106$$

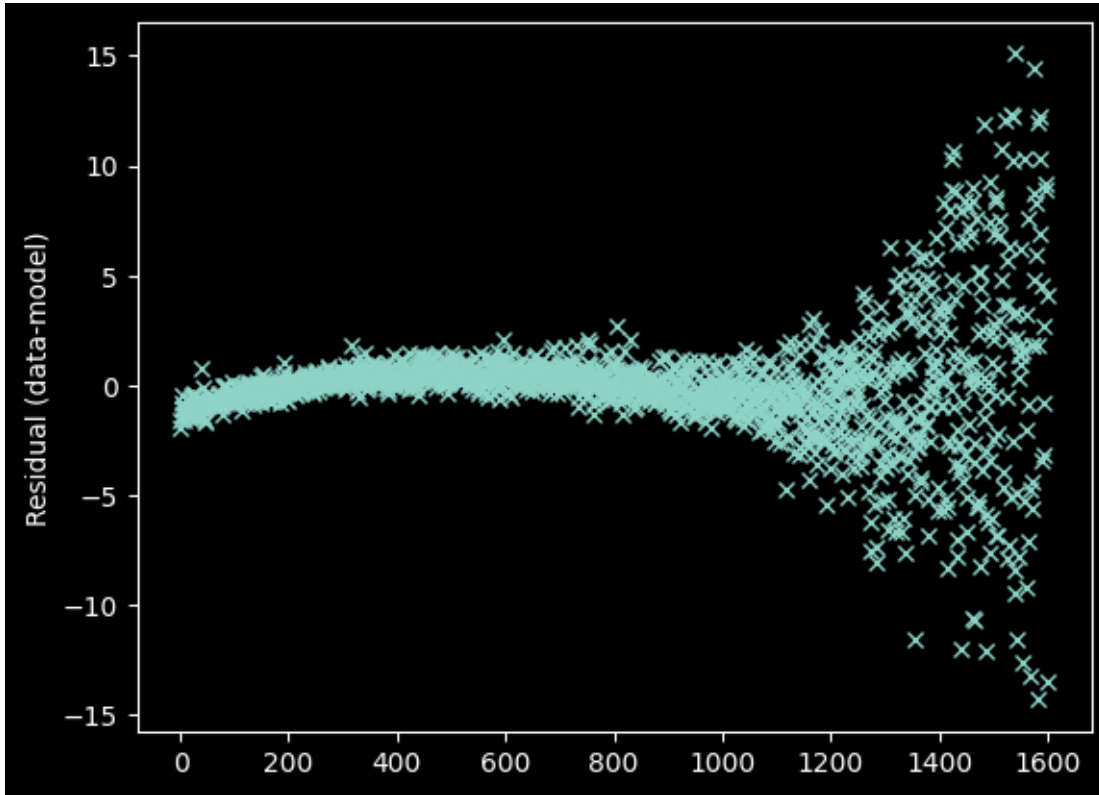
Este código ajusta un polinomio de segundo grado a un conjunto de datos filtrado, utilizando mínimos cuadrados, y luego grafica los datos originales junto con la curva ajustada.

```
[24]: plt.plot(xvals[~bad_moments], weighted_yaxis_values[~bad_moments], 'x', alpha=0.
↪ 5)
plt.plot(xvals, fitted_polymodel(xvals), color='r');
```



Graficamos y examinamos los residuos para inspeccionar visualmente si el ajuste es bueno:

```
[25]: plt.plot(xvals[~bad_moments],
              weighted_yaxis_values[~bad_moments] -
              ↪fitted_polymodel(xvals[~bad_moments]), 'x')
plt.ylabel("Residual (data-model)");
```



La curvatura que se observa a la izquierda indica un ajuste subóptimo. En concreto, la curvatura del residuo indica que necesitamos usar un modelo de orden superior; es decir, necesitamos más términos en el polinomio. Cambiamos grado=2 a grado=3.

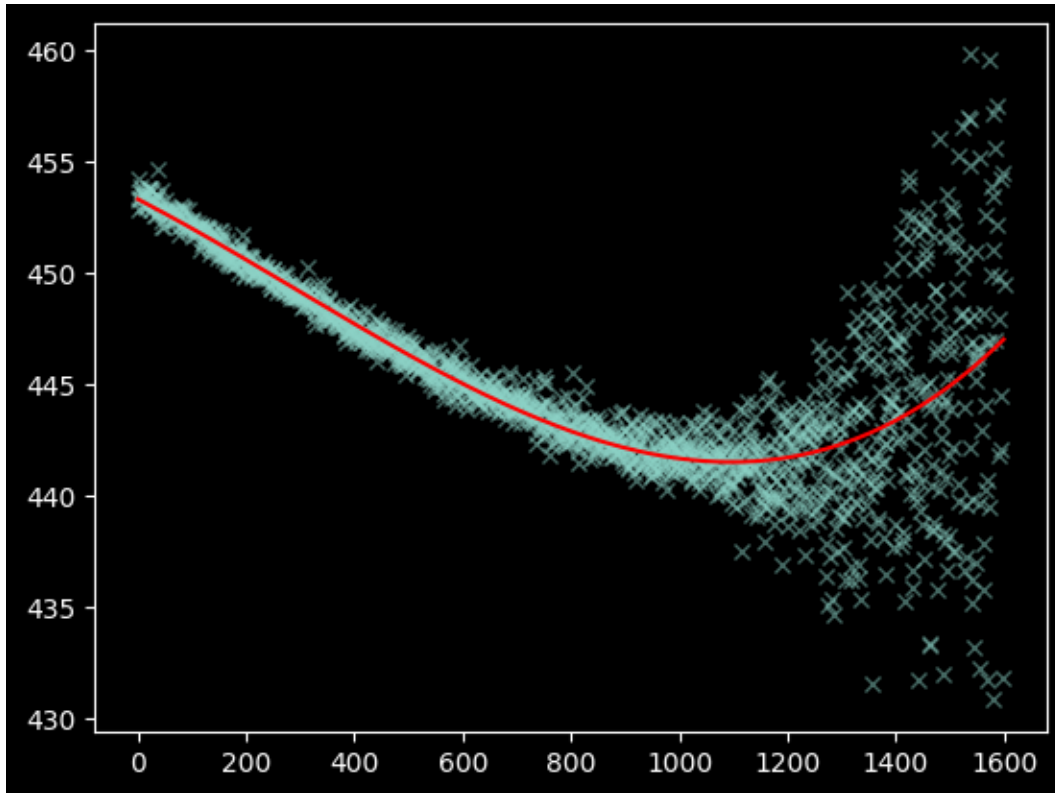
```
[26]: polymodel = Polynomial1D(degree=3)
fitted_polymodel = linfitter(polymodel, xvals[~bad_moments],
↪weighted_yaxis_values[~bad_moments])
fitted_polymodel
```

```
[26]: <Polynomial1D(3, c0=453.31622273, c1=-0.01256666, c2=-0.00000651,
c3=0.00000001)>
```

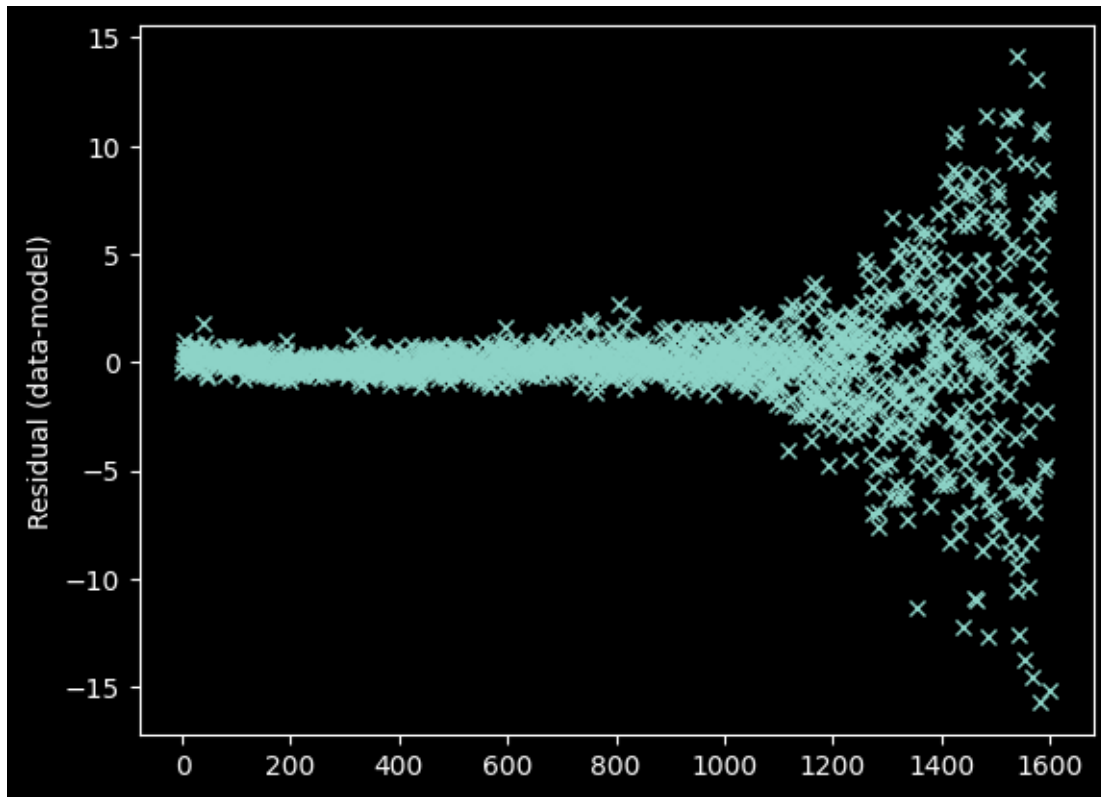
$$f(x) = 0.00000001x^3 - 0.00000651x^2 - 0.01256666x + 453.31622273$$

```
[27]: plt.plot(xvals[~bad_moments], weighted_yaxis_values[~bad_moments], 'x', alpha=0.
↪5)
```

```
plt.plot(xvals, fitted_polymodel(xvals), color='r');
```

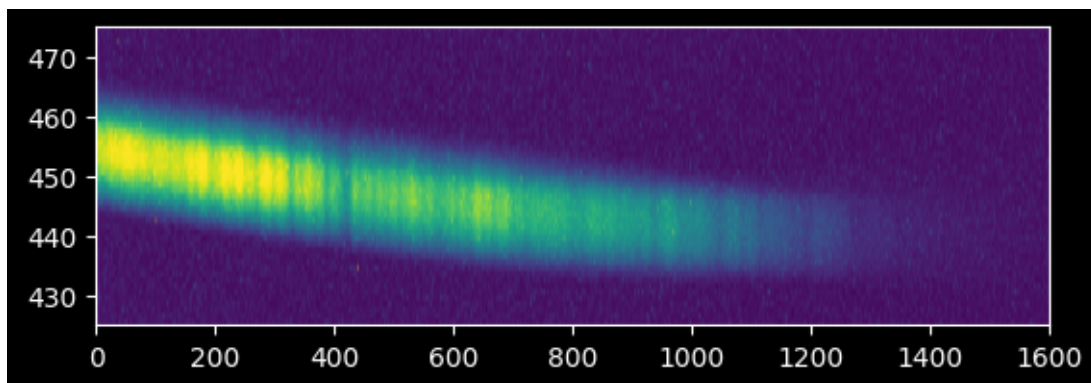


```
[28]: plt.plot(xvals[~bad_moments],  
              weighted_yaxis_values[~bad_moments] -  
              ↪fitted_polymodel(xvals[~bad_moments]), 'x')  
plt.ylabel("Residual (data-model)");
```

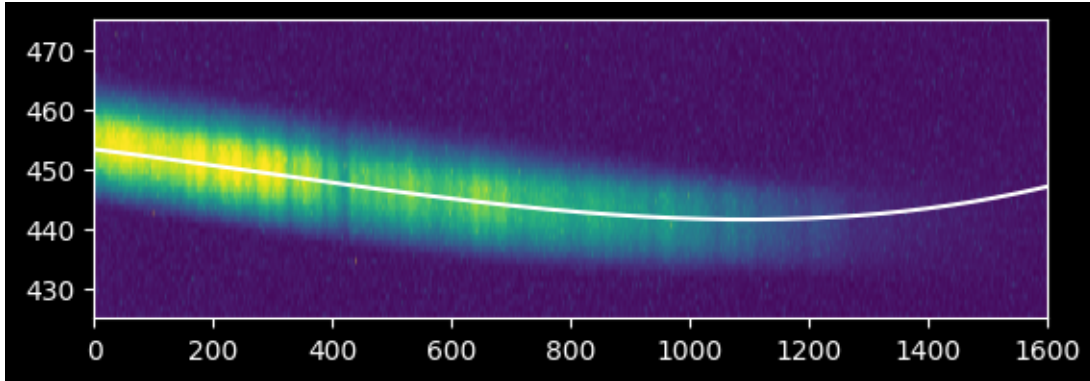
Podríamos descartar los datos a más de 1400 píxeles, ya que no hay una señal clara allí. Volveremos a esto...

```
[29]: plt.imshow(image_array[425:475,:],
                extent=[0,image_array.shape[1],425,475])
plt.gca().set_aspect(10);
```



Nuevamente, debemos verificar el rastro superponiendo el gráfico sobre los datos originales:

```
[30]: plt.imshow(image_array[425:475,:], extent=[0,image_array.shape[1],425,475])
plt.gca().set_aspect(10)
plt.plot(xvals, fitted_polymodel(xvals), 'w')
plt.axis((0,1600,425,475));
```



Al observar la curva hacia arriba en el modelo hacia la derecha (que no observamos en los datos), se sugiere que deberíamos volver a ajustar sin incluir en absoluto los datos $x > 1200$:

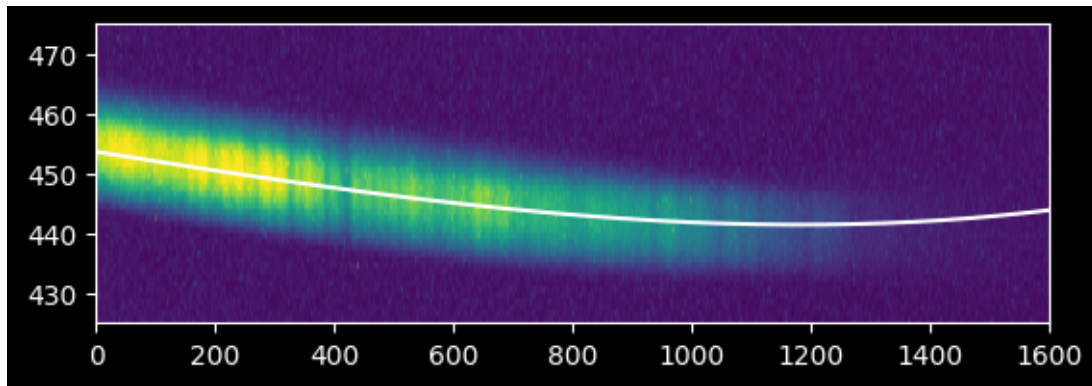
```
[31]: polymodel = Polynomial1D(degree=3)
fitted_polymodel = linfitter(polymodel, xvals[ (~bad_moments) & (xvals < 1200)],
                                weighted_yaxis_values[ (~bad_moments) & (xvals <
↪1200)])
fitted_polymodel
```

```
[31]: <Polynomial1D(3, c0=453.63074008, c1=-0.01596396, c2=0.00000083, c3=0.)>
```

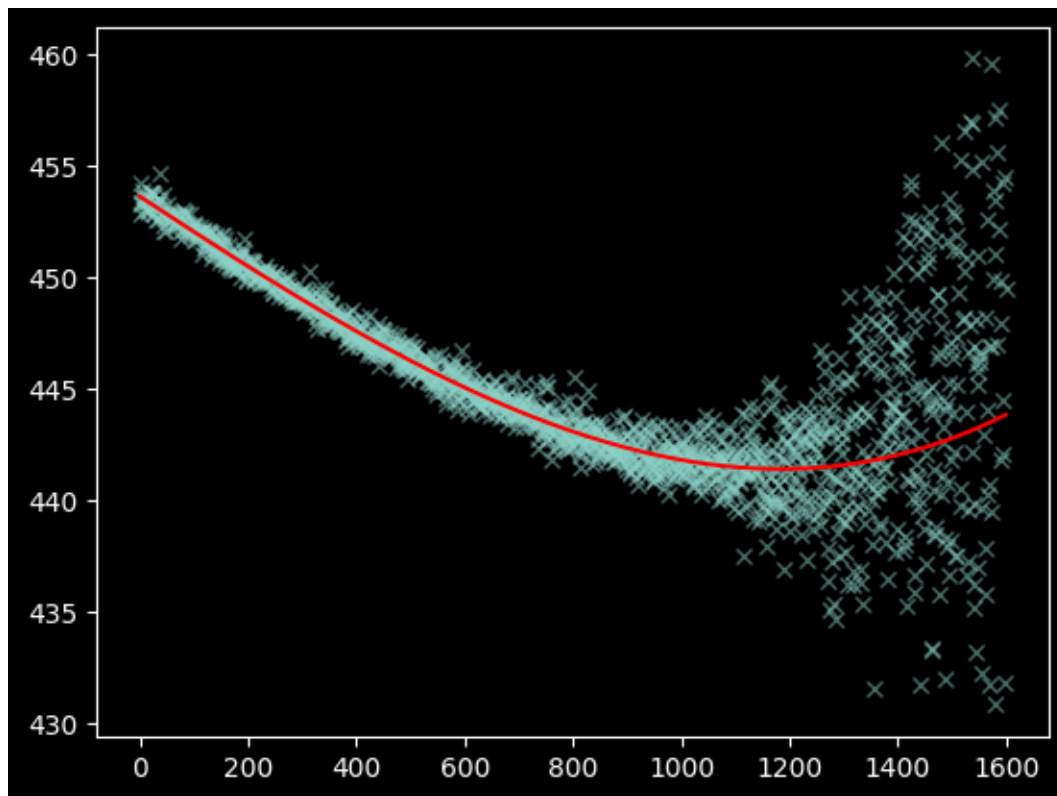
$$f(x) = 0.x^3 + 0.00000083x^2 - 0.01596396x + 453.63074008$$

Ahora tenemos un ajuste satisfactorio:

```
[32]: plt.imshow(image_array[425:475,:], extent=[0,image_array.shape[1],425,475])
plt.gca().set_aspect(10)
plt.plot(xvals, fitted_polymodel(xvals), 'w')
plt.axis((0,1600,425,475));
```



```
[33]: plt.plot(xvals[~bad_moments], weighted_yaxis_values[~bad_moments], 'x', alpha=0.5)
plt.plot(xvals, fitted_polymodel(xvals), color='r');
```

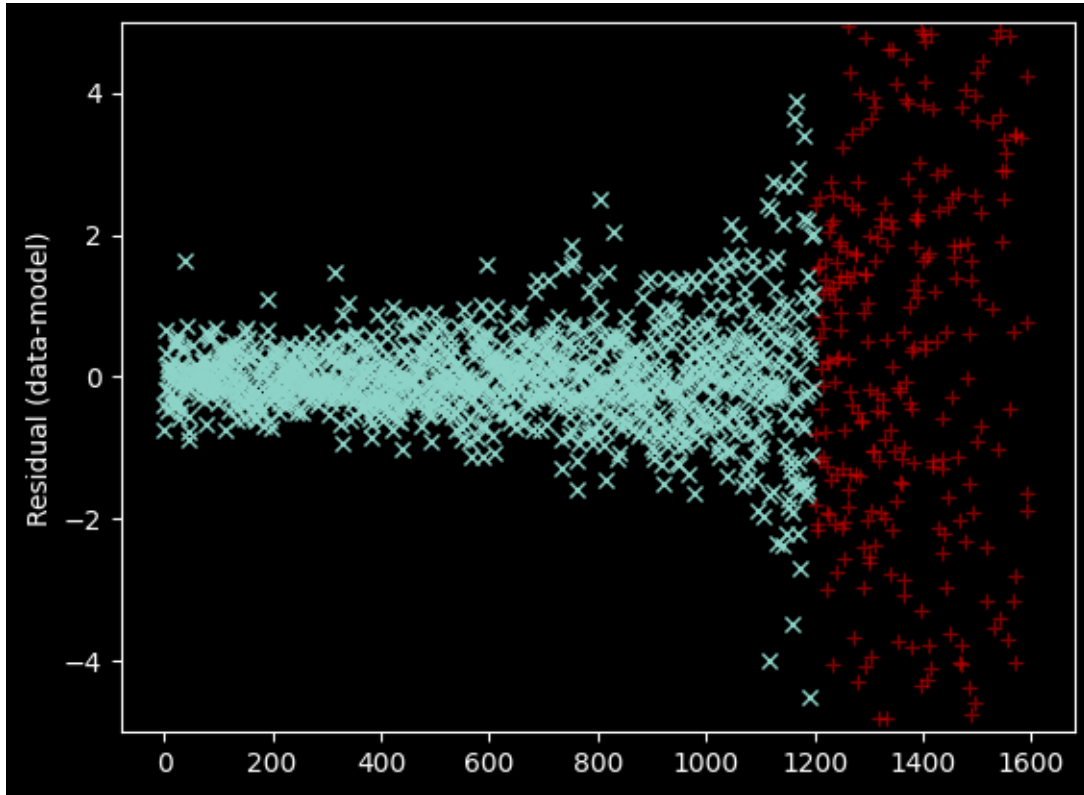


```
[34]: plt.plot(xvals[~bad_moments & (xvals < 1200)],
              weighted_yaxis_values[~bad_moments & (xvals < 1200)] -
              fitted_polymodel(xvals[~bad_moments & (xvals < 1200)]), 'x')
plt.plot(xvals[~bad_moments & (xvals > 1200)],
```

```

        weighted_yaxis_values[~bad_moments & (xvals > 1200)] -=
        ↪fitted_polymodel(xvals[~bad_moments & (xvals > 1200)]), 'r+', alpha=0.5)
plt.ylim(-5, 5)
plt.ylabel("Residual (data-model)");

```



2.2 Paso 4 : Crear un perfil de la traza.

Ahora podemos extraer los datos a lo largo de esa traza.

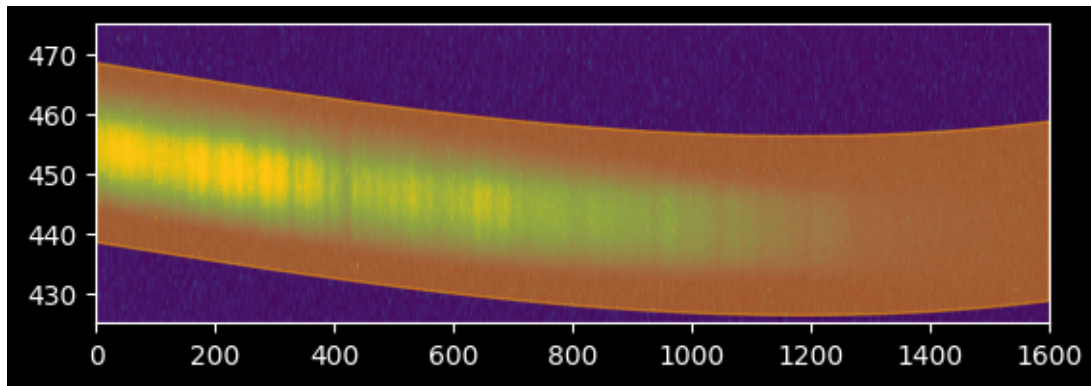
Queremos crear un perfil de la traza para ver cuántos píxeles a cada lado de la línea debemos incluir.

Este fragmento de código usa matplotlib para visualizar una porción de una imagen y superponer una región sombreada alrededor de un modelo polinomial ajustado.

```

[35]: plt.imshow(image_array[425:475,:], extent=[0,image_array.shape[1],425,475])
plt.gca().set_aspect(10)
plt.fill_between(xvals, fitted_polymodel(xvals)-15,
                 fitted_polymodel(xvals)+15,
                 color='orange', alpha=0.5)
plt.axis((0,1600,425,475));

```



Ese último paso merece una explicación:

`cutouts = np.array([image_array[int(yval)-npixels_to_cut:int(yval)+npixels_to_cut, ii] for yval, ii in zip(trace_center, xvals)])` [...] `for yval, ii in zip(trace_center, xvals)` toma cada valor de traza `y` y cada valor de traza `x` y los comprime, por lo que cada iteración del bucle `for` tiene un par `x, y`. `image_array[int(yval)-npixels_to_cut:int(yval)+npixels_to_cut, ii]` toma un solo píxel en la dirección `x` (la segunda dimensión, `ii`) y un rango de píxeles en la dirección `y`, es decir, $y \pm n$. Estos se unen en un bucle, por lo que tenemos una región de píxeles $y \pm n$ para cada uno. `x` píxeles. Luego los convertimos en una matriz. Podemos ver el resultado visualmente:

```
[36]: npixels_to_cut = 15
      trace_center = fitted_polymodel(xvals)
      cutouts = np.array([image_array[int(yval)-npixels_to_cut:
      ↪int(yval)+npixels_to_cut, ii]
                          for yval, ii in zip(trace_center, xvals)])
      cutouts.shape
```

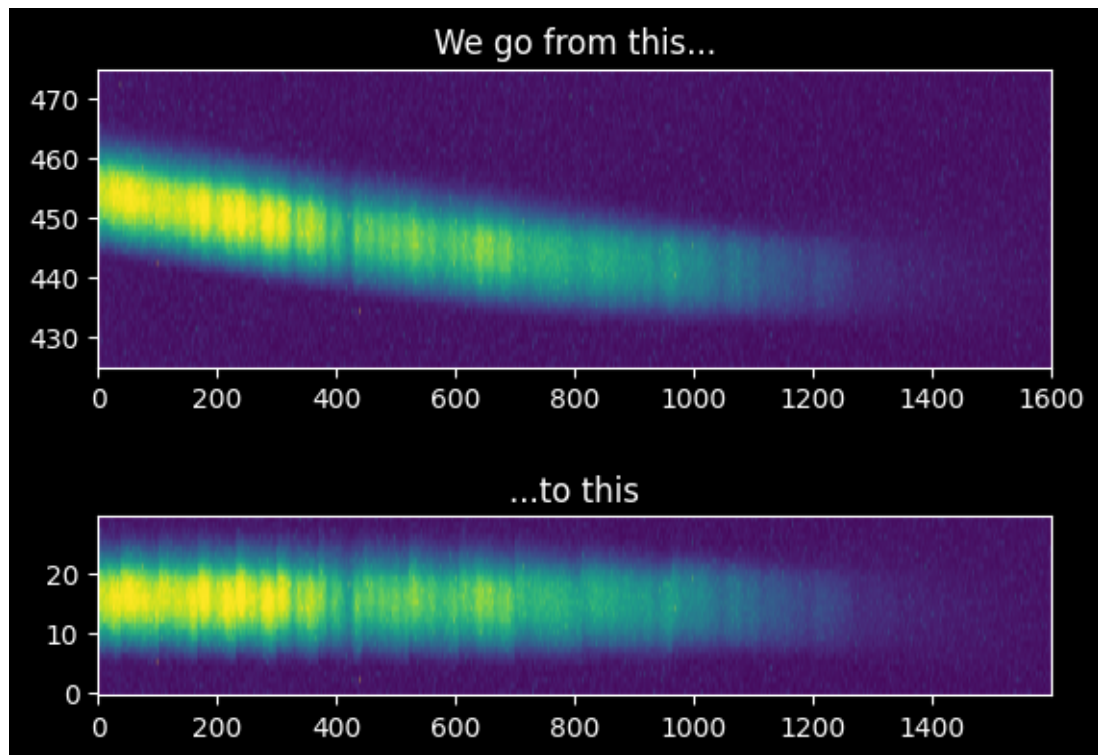
[36]: (1600, 30)

Este fragmento de código extrae pequeñas porciones de la imagen a lo largo de una curva ajustada (definida por `fitted_polymodel(xvals)`). Extrae pequeñas secciones verticales de la imagen a lo largo de la curva `fitted_polymodel(xvals)`. Este código muestra cómo se transforma una franja de la imagen original en una versión ajustada basada en `cutouts`.

Primer gráfico: Porción original de la imagen.

Segundo gráfico: Porción procesada y alineada con la curva del modelo polinomial.

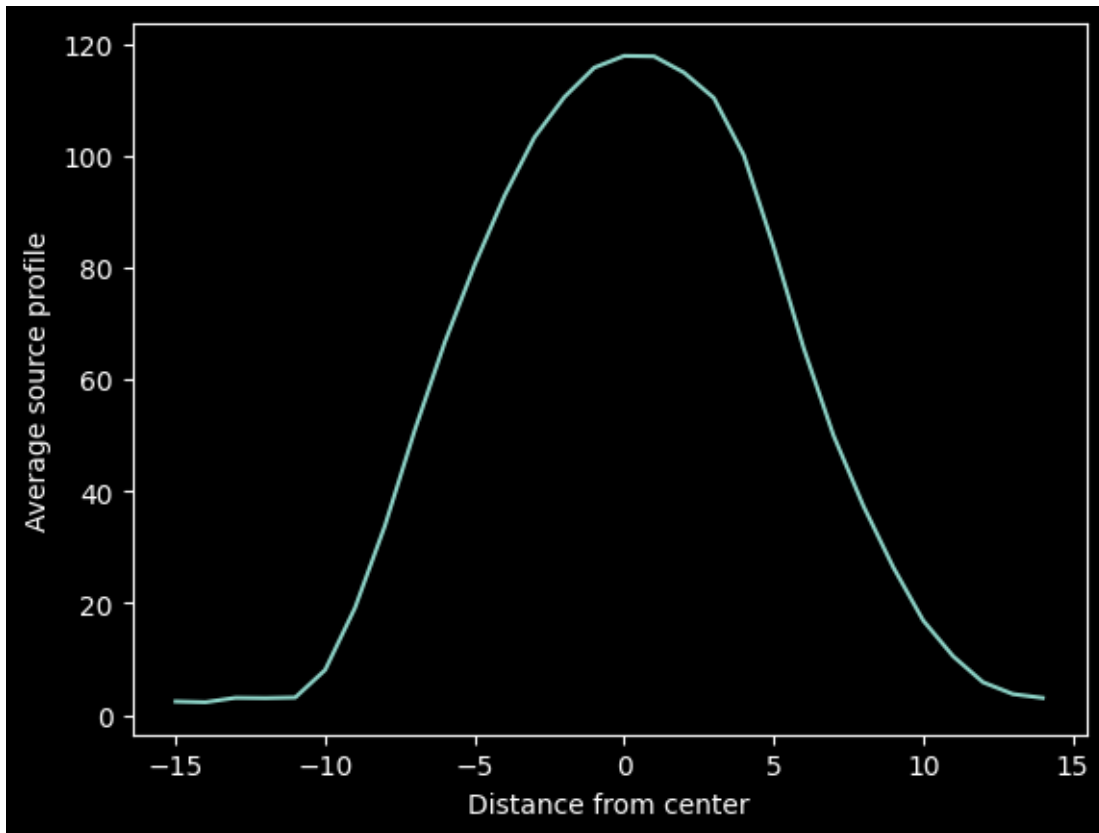
```
[37]: ax1 = plt.subplot(2,1,1)
      ax1.imshow(image_array[425:475,:], extent=[0,image_array.shape[1],425,475])
      ax1.set_aspect(10)
      ax1.set_title("We go from this...")
      ax2 = plt.subplot(2,1,2)
      ax2.imshow(cutouts.T)
      ax2.set_title("...to this")
      ax2.set_aspect(10)
```



Luego, promediamos a lo largo de la dirección X para obtener el perfil de traza:

Este código calcula y visualiza el perfil promedio de la traza extraída de la imagen.

```
[38]: mean_trace_profile = (cutouts - background).mean(axis=0)
      trace_profile_xaxis = np.arange(-npixels_to_cut, npixels_to_cut)
      plt.plot(trace_profile_xaxis, mean_trace_profile)
      plt.xlabel("Distance from center")
      plt.ylabel("Average source profile");
```



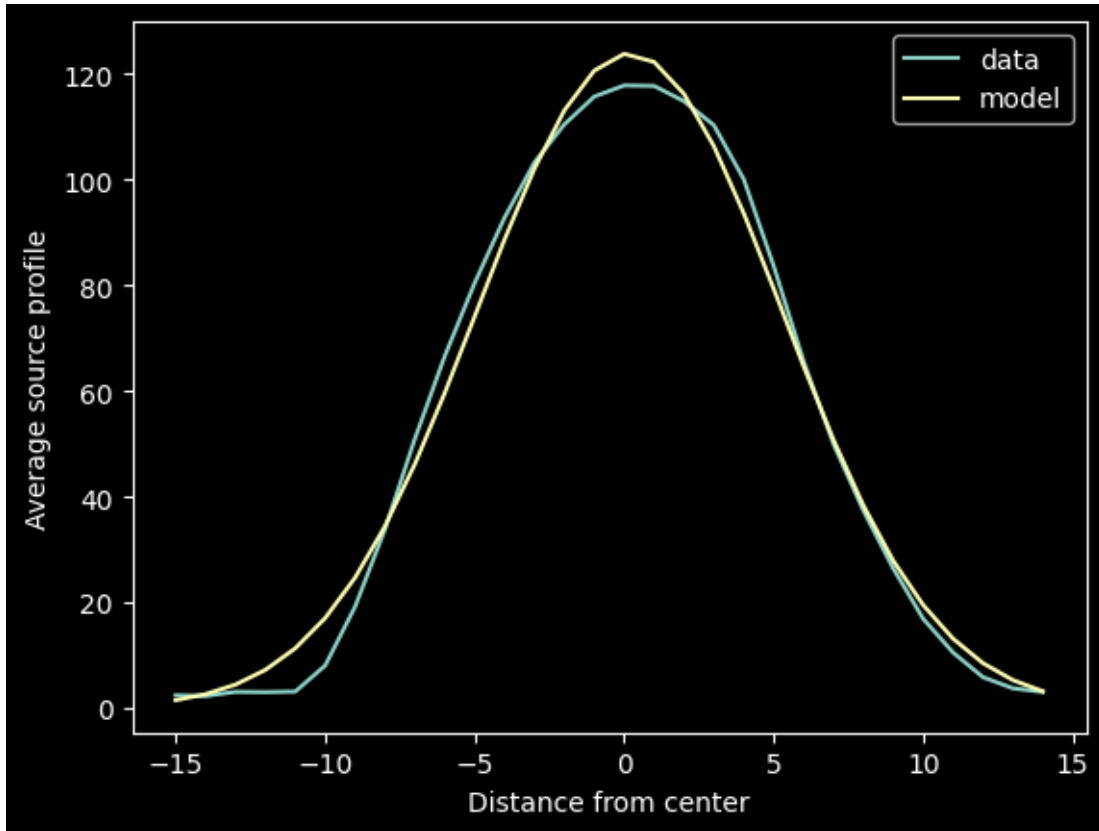
Queremos ajustar ese perfil con un gaussiano para uso futuro, por lo que importamos el perfil del modelo gaussiano y el ajustador no lineal y ejecutamos un ajuste:

```
[42]: from astropy.modeling.models import Gaussian1D
      from astropy.modeling.fitting import LevMarLSQFitter

      lmfitter = LevMarLSQFitter()
      guess = Gaussian1D(amplitude=mean_trace_profile.max(), mean=0, stddev=5)
      fitted_trace_profile = lmfitter(model=guess, x=trace_profile_xaxis,
      ↪ y=mean_trace_profile)
      model_trace_profile = fitted_trace_profile(trace_profile_xaxis)
      fitted_trace_profile
```

```
[42]: <Gaussian1D(amplitude=123.84846797, mean=0.17719819, stddev=5.10872134)>
```

```
[43]: plt.plot(trace_profile_xaxis, mean_trace_profile, label='data')
      plt.plot(trace_profile_xaxis, model_trace_profile, label='model')
      plt.legend(loc='best')
      plt.xlabel("Distance from center")
      plt.ylabel("Average source profile");
```

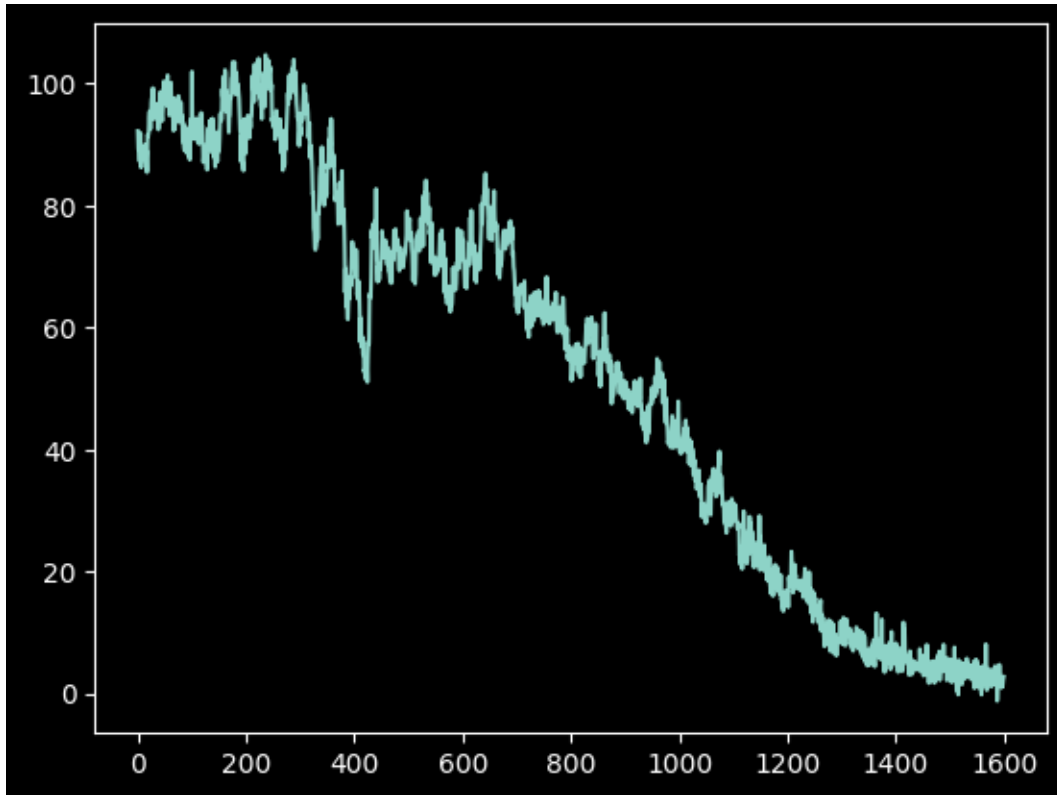


Se pueden utilizar razonablemente tanto el perfil de seguimiento empírico `mean_trace_profile` como el `model_trace_profile` modelado; este último es más conveniente para serializar (es decir, escribir en disco o en papel).

3 Paso 5 : Sacar espectro de la traza

Podemos obtener nuestro espectro promediando directamente los píxeles a lo largo de la traza:

```
[44]: average_spectrum = (cutouts - background).mean(axis=1)
      plt.plot(average_spectrum);
```

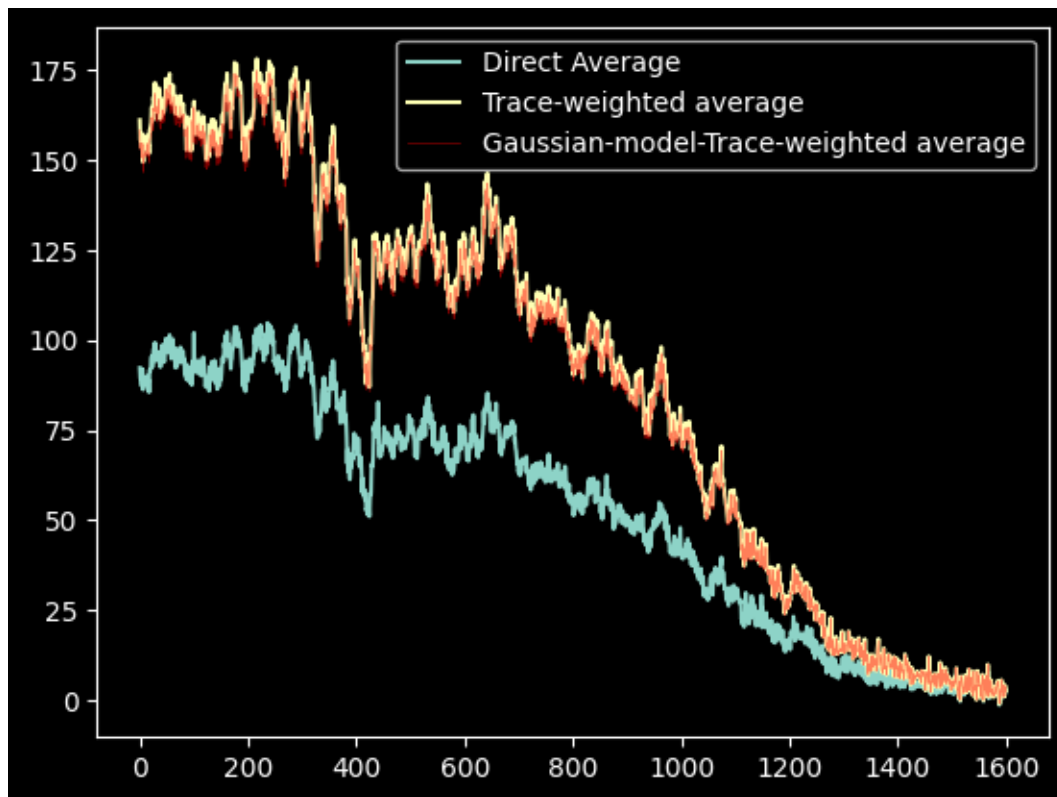
O podemos obtener nuestro espectro tomando el promedio ponderado por trazas:

```
[45]: trace_avg_spectrum = np.array([np.average(
    image_array[int(yval)-npixels_to_cut:int(yval)+npixels_to_cut, ii] -
    ↪background,
    weights=mean_trace_profile)
    for yval, ii in zip(trace_center, xvals)])
```

También podemos hacer esto con los pesos gaussianos:

```
[46]: gaussian_trace_avg_spectrum = np.array([np.average(
    image_array[int(yval)-npixels_to_cut:int(yval)+npixels_to_cut, ii] -
    ↪background,
    weights=model_trace_profile)
    for yval, ii in zip(trace_center, xvals)])
```

```
[47]: plt.plot(average_spectrum, label="Direct Average")
plt.plot(trace_avg_spectrum, label='Trace-weighted average')
plt.plot(gaussian_trace_avg_spectrum, label='Gaussian-model-Trace-weighted_
    ↪average', alpha=0.5, linewidth=0.5, color='r')
plt.legend(loc='best');
```



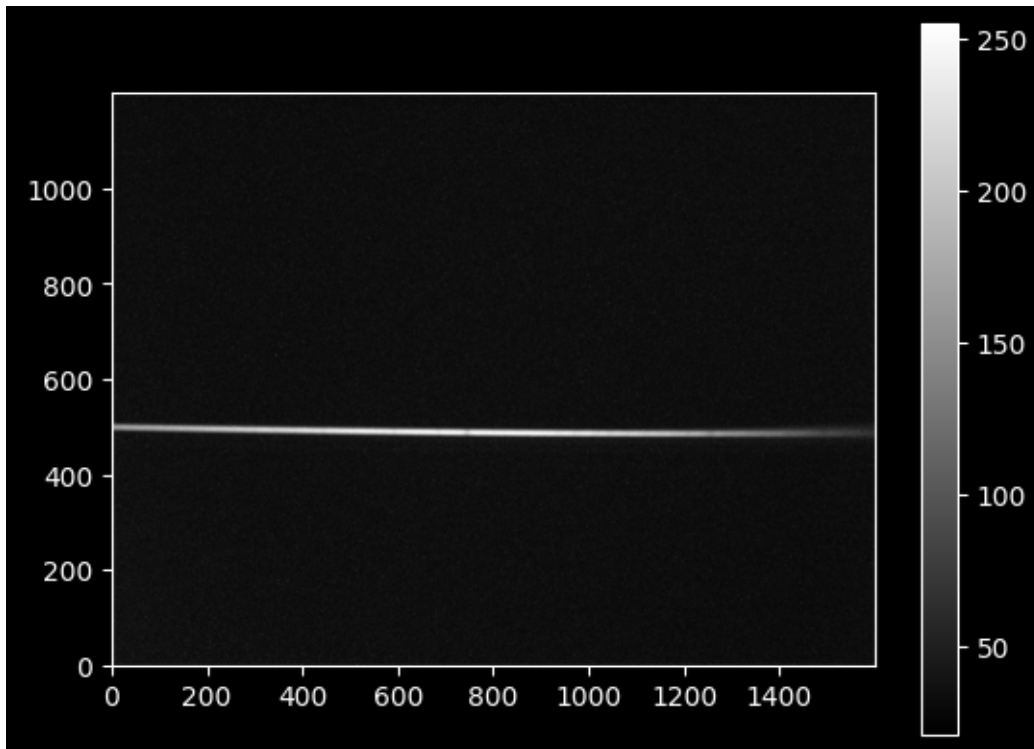
En general, el promedio ponderado por trazas tendrá una relación señal-ruido más alta, como se observa aquí (aunque no hemos medido el ruido, este es aproximadamente constante en toda la imagen).

Observe que el modelo gaussiano y la traza directa arrojan resultados casi idénticos.

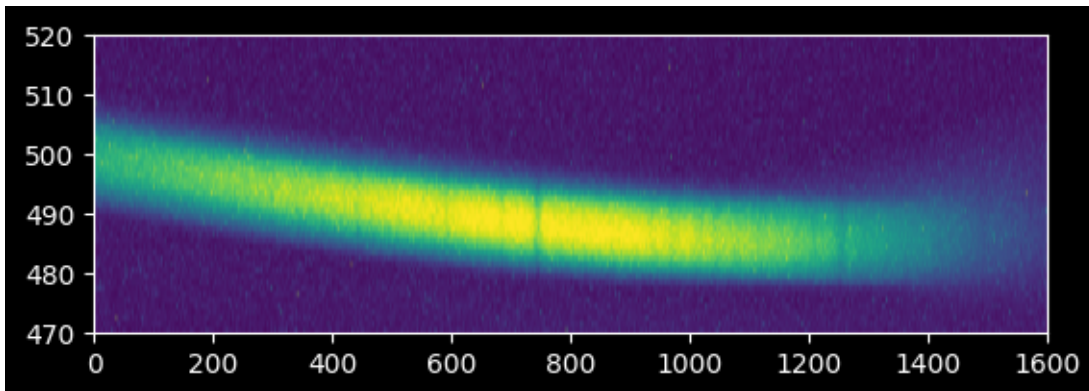
4 Paso 6: Repita para otra estrella

En este último paso, repasamos todos los pasos anteriores nuevamente para otra estrella (Deneb), pero con menos explicación.

```
[48]: image_array_2 = np.array(Image.open('deneb_3s_13.63g_1.bmp'))  
      plt.imshow(image_array_2, cmap='gray')  
      plt.colorbar();
```

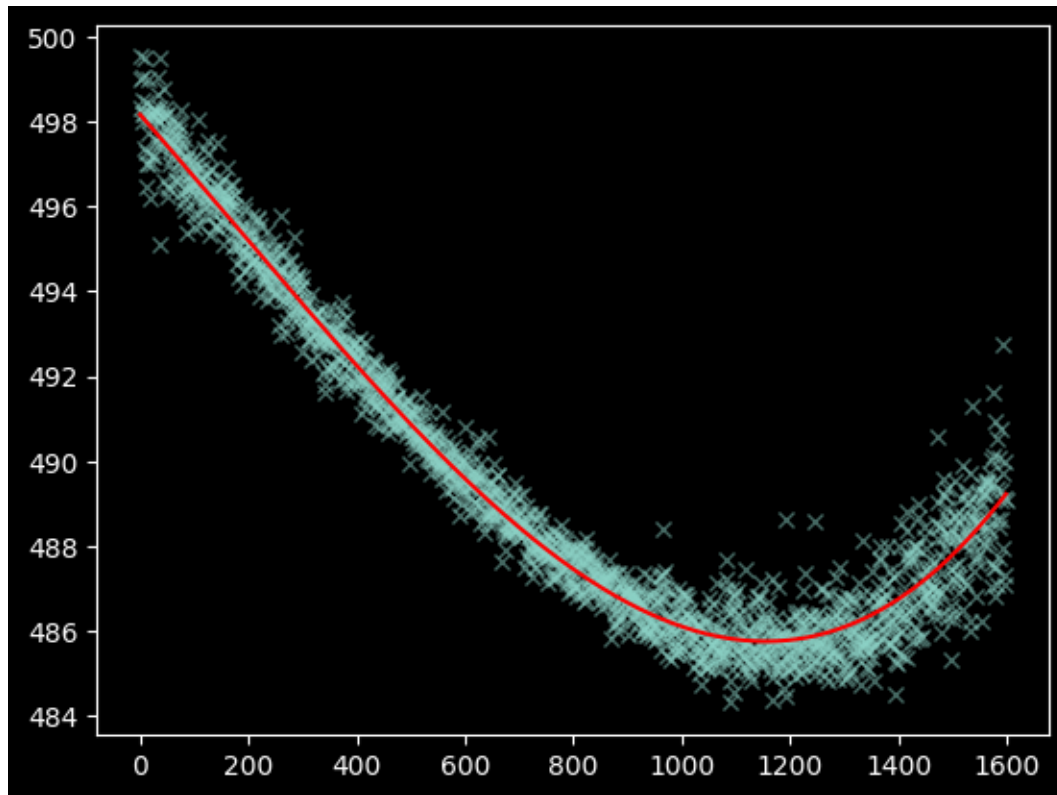


```
[49]: plt.imshow(image_array_2[470:520,:], extent=[0,1600,470,520])
      plt.gca().set_aspect(10)
```

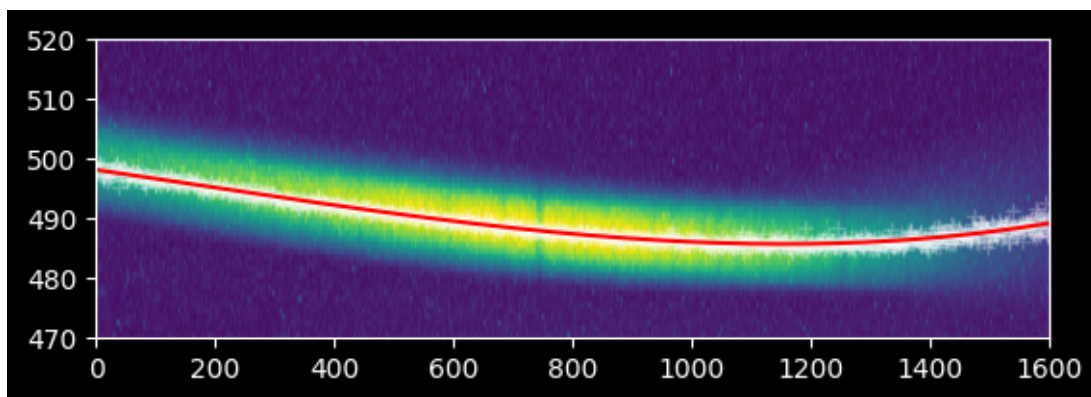


```
[ ]: yaxis2 = np.repeat(np.arange(470, 520)[: ,None], image_array_2.shape[1], axis=1)
      weighted_yaxis_values2 = np.average(yaxis2, axis=0, weights=image_array_2[470:
      ↪ 520,:]) - np.median(image_array_2)
      polymodel2 = Polynomial1D(degree=3)
      fitted_polymodel2 = linfitter(polymodel2, xvals, weighted_yaxis_values2)
      trace_center2 = fitted_polymodel2(xvals)
```

```
[ ]: plt.plot(xvals, weighted_yaxis_values2, 'x', alpha=0.5)
plt.plot(xvals, trace_center2, color='r');
```

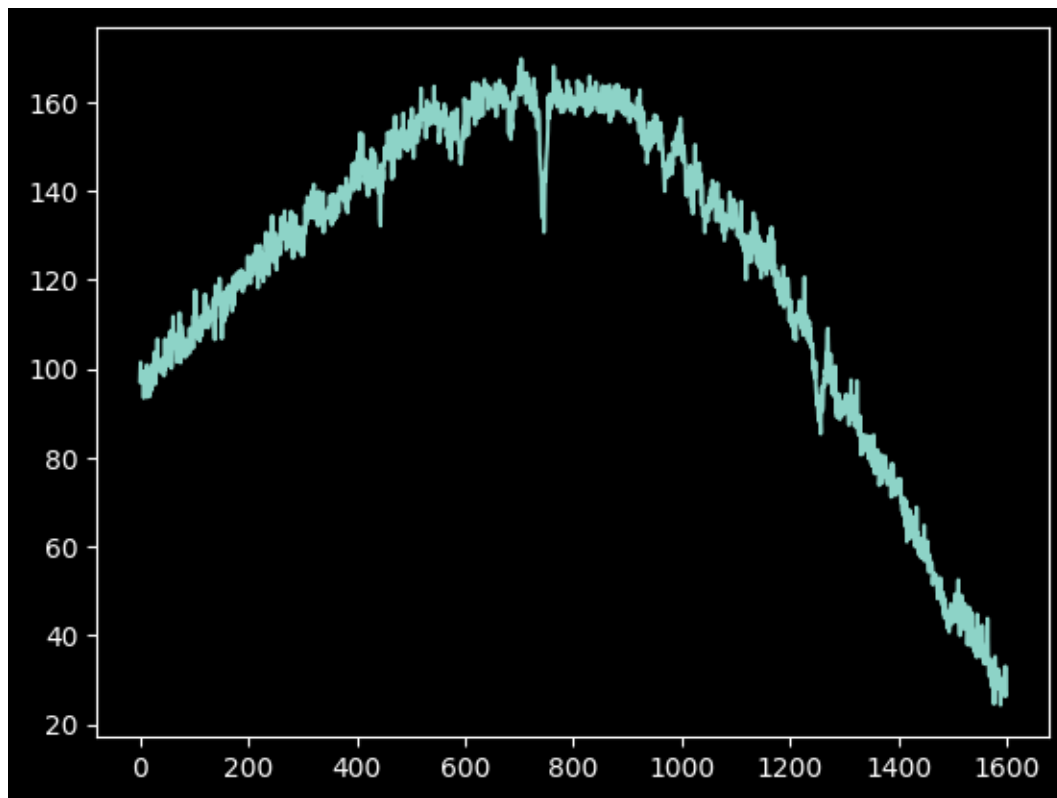


```
[ ]: plt.imshow(image_array_2[470:520,:], extent=[0,1600,470,520])
plt.plot(xvals, weighted_yaxis_values2, 'w+', alpha=0.25)
plt.plot(xvals, trace_center2, color='r')
plt.gca().set_aspect(10)
```



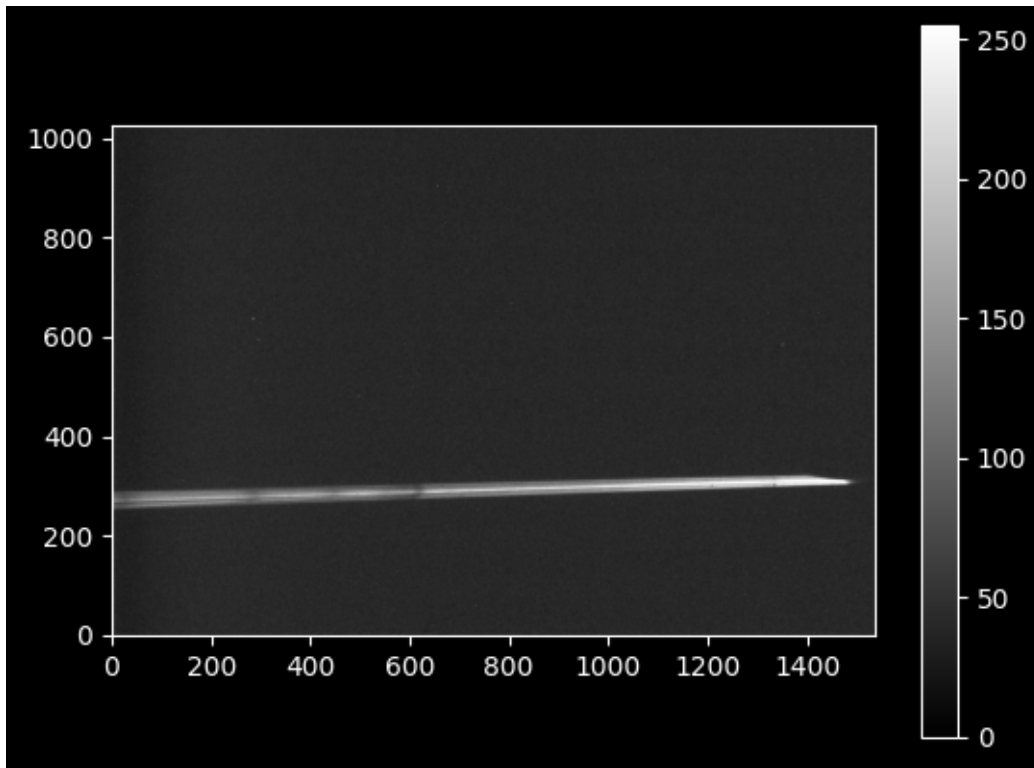
```
[ ]: spectrum2 = np.array([np.average(image_array_2[int(yval)-npixels_to_cut:
    ↪int(yval)+npixels_to_cut, ii] - np.median(image_array_2),
    weights=mean_trace_profile)
    for yval, ii in zip(trace_center2, xvals)])
```

```
[ ]: plt.plot(spectrum2);
```

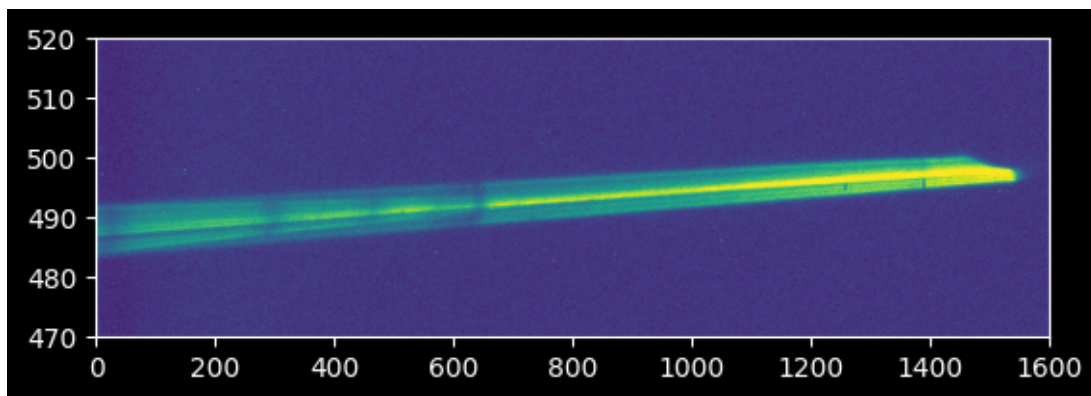


En el próximo tutorial, Reducción de datos espectroscópicos 2, trabajaremos en la calibración de la longitud de onda.

```
[51]: image_array_3 = np.array(Image.open('vega_2.jpeg').convert("L"))
plt.imshow(image_array_3, cmap='gray')
plt.colorbar();
```



```
[52]: plt.imshow(image_array_3[200:400,:], extent=[0,1600,470,520])
plt.gca().set_aspect(10)
```



```
[ ]:
```

```
[55]: xvals = np.arange(image_array_3.shape[1])
yaxis3 = np.repeat(np.arange(200, 400)[: ,None], image_array_3.shape[1], axis=1)
```

```

weighted_yaxis_values3 = np.average(yaxis3, axis=0, weights=image_array_3[200:
↪400,:]) - np.median(image_array_3))
polymodel3 = Polynomial1D(degree=3)
fitted_polymodel3 = linfitter(polymodel3, xvals, weighted_yaxis_values3)
trace_center3 = fitted_polymodel3(xvals)

```

```

[59]: spectrum3 = np.array([np.average(image_array_3[int(yval)-npixels_to_cut:
↪int(yval)+npixels_to_cut, ii]) - np.median(image_array_3),
                             weights=mean_trace_profile)
      for yval, ii in zip(trace_center3, xvals)])

```

```

[60]: plt.plot(spectrum3);

```

